Project Report

Search Typeahead (Autocomplete) System
Distributed cache · consistent hashing · trending · batch writes

Author: Shubh Srivastava | Date: 22 June 2026

Overview. A search typeahead system that suggests popular queries as you type, records submitted searches, and serves suggestions with low latency from a distributed in-memory cache routed by consistent hashing. It supports trending searches (recency-aware ranking) and batch writes so the database is not hammered on every keystroke. The stack is a Node.js / Express backend, a SQLite primary store, and a React (Vite) front end.

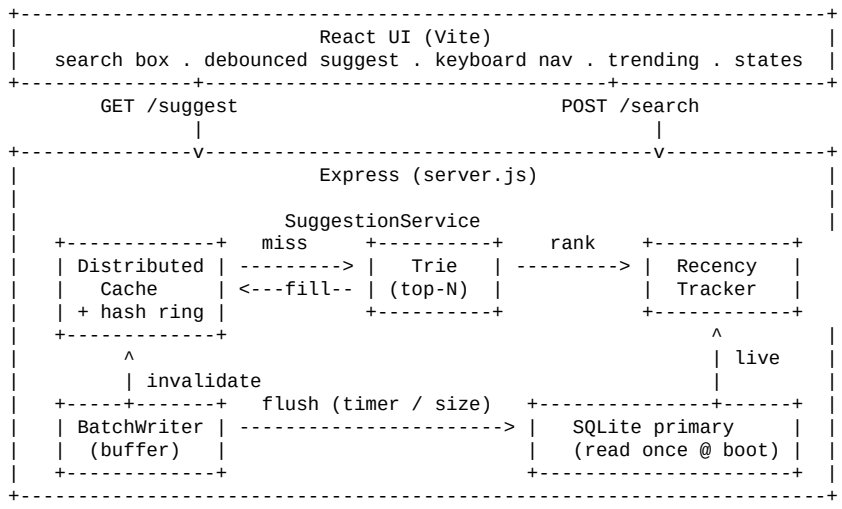
1. Architecture

The system is split into a thin HTTP layer and a set of focused in-memory components orchestrated by a single SuggestionService. Suggestion reads are served from memory (cache and trie); the SQLite database is read only once at startup and written only in batches.

1.1 Components

Component	File	Responsibility
Express server	server.js	HTTP routes; serves the built React UI
SuggestionService	suggestionService.js	Orchestrates read / write / flush paths
Trie	trie.js	Prefix index; each node caches its top-N queries
DistributedCache	distributedCache.js	N logical cache nodes (TTL + LRU)
ConsistentHashRing	consistentHash.js	Maps prefix key -> owning cache node
RecencyTracker	recency.js	Time-decayed scores -> trending + recency rank
BatchWriter	batchWriter.js	Buffers + aggregates counts, flushes in batches
SQLite store	db.js	Durable query->count store (read once, batched writes)

1.2 Component diagram



1.3 Data flows

Read — GET /suggest?q=iph&mode=count

- Normalize the prefix (lowercase / trim).
- Build cache key `sug:<mode>:<prefix>`; the hash ring picks the owning node.
- Cache hit -> return stored suggestions (the common case).
- Cache miss -> read the candidate pool from the trie node for that prefix, rank it (by count, or by the popularity+recency blend), keep the top 10, store in the cache with TTL, and return.

Write — POST /search

- Normalize the query.

- Update the in-memory live count so the next flush knows the new total.
- Record into the RecencyTracker (trending updates immediately).
- Add to the BatchWriter buffer and return at once — no synchronous DB write.

Flush — every 2s or 500 buffered increments

- Aggregate buffered deltas into one SQLite transaction (`+delta` upserts).
- Refresh the trie's top-N pools for affected queries.
- Invalidate cache keys for every prefix of each affected query (both modes).

2. Dataset source and loading

2.1 Source

- **Source:** generated locally by `backend/src/seed.js`. The spec allows any dataset and permits deriving counts by aggregation.
- **How counts are derived:** queries are built by combining real brand / product / tech vocabulary across several query templates, then assigned counts from a Zipf-like (power-law) distribution — a few head queries are searched enormously, a long tail barely at all. This matches how real search traffic is shaped.
- **Reproducible:** a fixed PRNG seed (mulberry32, seed 42) means every run produces the exact same dataset.
- **Size:** 120,000 unique queries (above the 100k minimum).
- **Output:** `backend/data/queries.csv` (columns `query,count`) and the SQLite DB `backend/data/typeahead.db`.

2.2 Loading instructions

Requirements: Node 18+ (tested on Node 20). Run from the project root:

```
npm install            # installs concurrently (for the dev script)
npm run install:all    # installs backend + frontend deps
npm run seed           # generate dataset (120k queries) + load SQLite

# Option A - one-command demo (backend serves the built UI on :3001)
npm run build          # build the React UI into frontend/dist
npm start              # http://localhost:3001

# Option B - dev mode (hot reload): backend :3001, UI :5173
npm run dev            # then open http://localhost:5173

# Measure performance (server must be running)
npm run benchmark
```

`npm run seed` writes the CSV and bulk-inserts into SQLite in one transaction. The server reads the table **once at startup** to build the in-memory trie; after that, suggestion reads never touch the DB.

3. API documentation

Base URL: `http://localhost:3001`

3.1 GET /suggest?q=<prefix>&mode=count|recency

Returns up to 10 prefix-matching suggestions sorted by ranking.

- q — the typed prefix (case-insensitive; empty / missing -> empty list).
- `mode` — `count` (basic, all-time popularity, default) or `recency` (enhanced, blends popularity with recent activity).

```
{
  "query": "iph", "mode": "count", "source": "cache",
  "cacheNode": "cache-node-0", "count": 10, "latencyMs": 0.18,
  "suggestions": [{ "query": "iphone", "count": 100001 }, ...]
}
```

`source` is `cache` (hit), `compute` (miss -> built from trie), or `empty`.

3.2 POST /search body { "query": "iphone" }

Records the search and returns a dummy response. Increments the count if the query exists, inserts it with an initial count if not.

```
{ "message": "Searched", "query": "iphone", "count": 100002 }
```

3.3 GET /cache/debug?prefix=<prefix>&mode=count|recency

Shows which cache node owns the prefix key and whether it is currently a hit or miss (non-mutating).

```
{ "prefix": "iph", "key": "sug:count:iph", "node": "cache-node-0", "status": "hit" }
```

3.4 GET /trending

Top trending searches by decayed recent activity.

```
{ "trending": [{ "query": "iphone", "recencyScore": 12.4, "count": 100050 }] }
```

3.5 GET /metrics

Cache hit rate (overall + per node), batch-write counters, DB read/write counts, consistent-hashing key distribution, and /suggest latency percentiles (p50 / p95 / p99). Used by the benchmark and for the performance report.

4. Design choices and trade-offs

Primary store — SQLite

A real on-disk DB so DB read/write counts are genuine. WAL + `synchronous=NORMAL` make batched writes fast while staying durable enough for the demo.

Suggestions — in-memory trie with a precomputed candidate pool

Each trie node caches the top-N (50) highest-count queries under that prefix, so /suggest is O(prefix length), never a full scan. The pool is bigger than the 10 we return so the recency re-ranker has room to promote a trending query. **Trade-off:** a query ranked below the 50th by count cannot be surfaced by recency for that prefix — bounded memory & latency vs. perfect freshness.

Cache — distributed, consistent-hashed

4 logical cache nodes; a consistent hash ring with 150 virtual nodes per node decides ownership. The same prefix always routes to the same node (affinity), and adding / removing a node only remaps ~1/N of keys. Each node has a TTL (30s) + an LRU bound so stale data cannot live forever. **Trade-off:** in-process logical nodes keep the demo runnable on one machine; in production each node would be a separate Redis / Memcached box behind the same (unchanged) ring logic.

Trending — time-decayed recency score

Each query keeps one exponentially-decayed counter (10-min half-life), updated live on every search, so trending feels real-time. Decay means a short viral spike fades on its own — this is exactly how the system avoids permanently over-ranking a briefly-popular query. Enhanced ranking blends: `score = 1.0 · log10(count+1) + 3.0 · recencyScore`.

Batch writes

Search submissions land in an in-memory buffer keyed by query (so 50 searches for "iphone" aggregate into one `+50`). The buffer flushes on a 2s timer **or** at 500 buffered increments, as a single SQLite transaction. **Failure trade-off:** the buffer is in memory, so a crash before a flush loses at most the last interval's increments. That is acceptable for popularity counters; if counts had to be exact we would add a write-ahead log before acknowledging.

Cache invalidation on ranking change

When a search is recorded and later flushed, the cached results for every prefix of that query are invalidated (in both modes) so the next read recomputes fresh rankings; TTL covers everything else.

5. Performance report

Measured locally with `npm run benchmark` (Node 20, 120k-query dataset, 4 cache nodes). Numbers vary by machine; reproduce with the command.

Metric	Result
Suggestion latency - warm (cache hit)	p50 ~ 0.11 ms, p95 ~ 0.5 ms (HTTP round-trip)
Suggestion latency - cold (compute)	p50 ~ 0.17 ms, p95 ~ 0.5 ms
Server-side /suggest compute	p50 0.003 ms, p95 0.008 ms
Cache hit rate (after warm-up)	~ 98%
Batch writes: 5,031 searches	-> 57 DB rows written across 13 flushes
Write reduction via batching	~ 98.9% fewer DB writes
Consistent-hashing distribution (676 keys / 4 nodes)	~ 181 / 184 / 153 / 158 (even)

Cache vs DB reads: suggestion reads are served entirely from the trie + cache; the DB is read only once at startup. So the DB read count stays flat regardless of query volume.

5.1 Basic vs. enhanced ranking

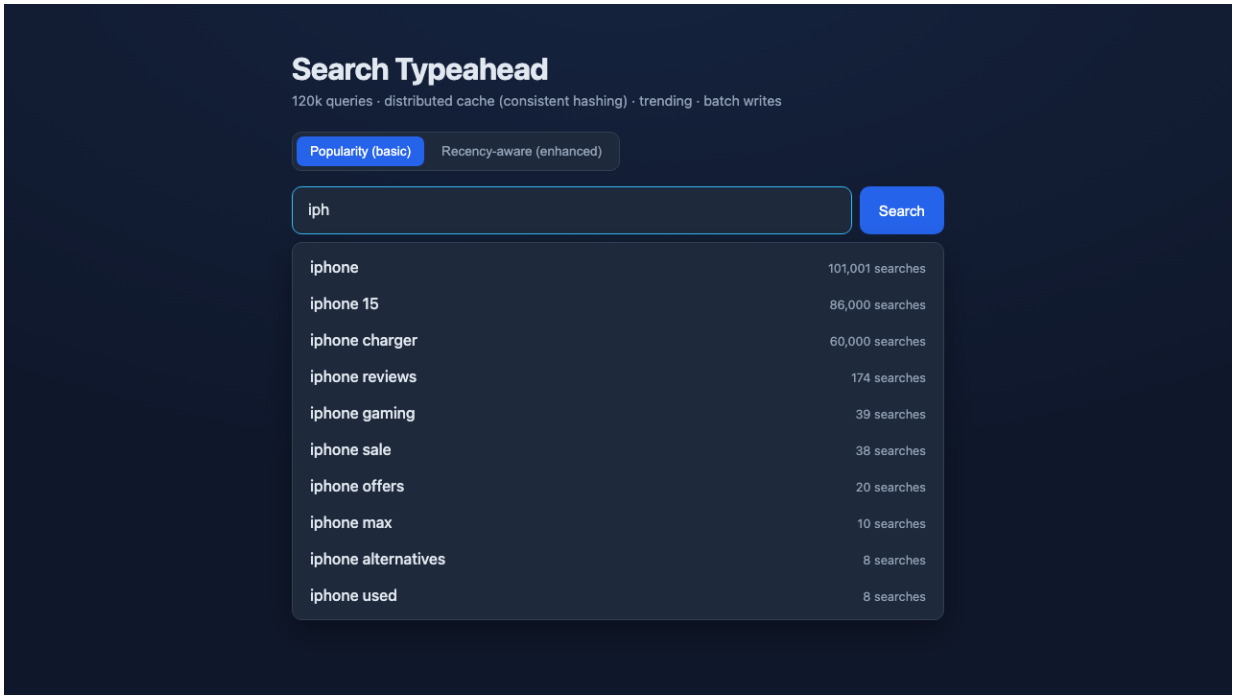
In the UI, toggle Popularity (basic) vs Recency-aware (enhanced), or via API:

```
# burst-search a low-count tail query
for i in $(seq 1 30); do curl -s -XPOST localhost:3001/search \
  -H 'content-type: application/json' -d '{"query":"iphone sale"}' >/dev/null; done
sleep 3
curl -s "localhost:3001/suggest?q=iph&mode=count"    # "iphone sale" stays near bottom
curl -s "localhost:3001/suggest?q=iph&mode=recency"  # "iphone sale" jumps to top, then decays
```

In recency mode a freshly-searched, low-count query is promoted to the top, while popularity mode keeps the all-time leaders first.

5.2 Screenshots

Popularity (basic) ranking



Recency-aware (enhanced) ranking